

PRESENTAT

GRUPO 2



Thành Viên

Phạm Gia

Khiêm

Trần Công

Quốc

Nguyễn Đăng

Khôi

Đình Thành

Đạt

Trần Thanh

Nhân

Đỗ Thiên

Phúc

074206004060

049206006580

079206030087

072206000894

080206012993

075206003542

MỤC LỤC

1 TỔNG QUAN KIẾN
THỨC

2 KIẾN TRÚC & CHI TIẾT DỰ
ÁN

3 TỔNG KẾT & HƯỚNG PHÁT
TRIỂN

Start It Now

1 TỔNG QUAN KIẾN THỨC

MÔ HÌNH CLIENT -SERVER
MÔ HÌNH P2P

Socket Programming

THƯ VIỆN
BOOST.ASIO

TCP - UDP

Asynchronous
Programming

MÔ HÌNH CLIENT -SERVER

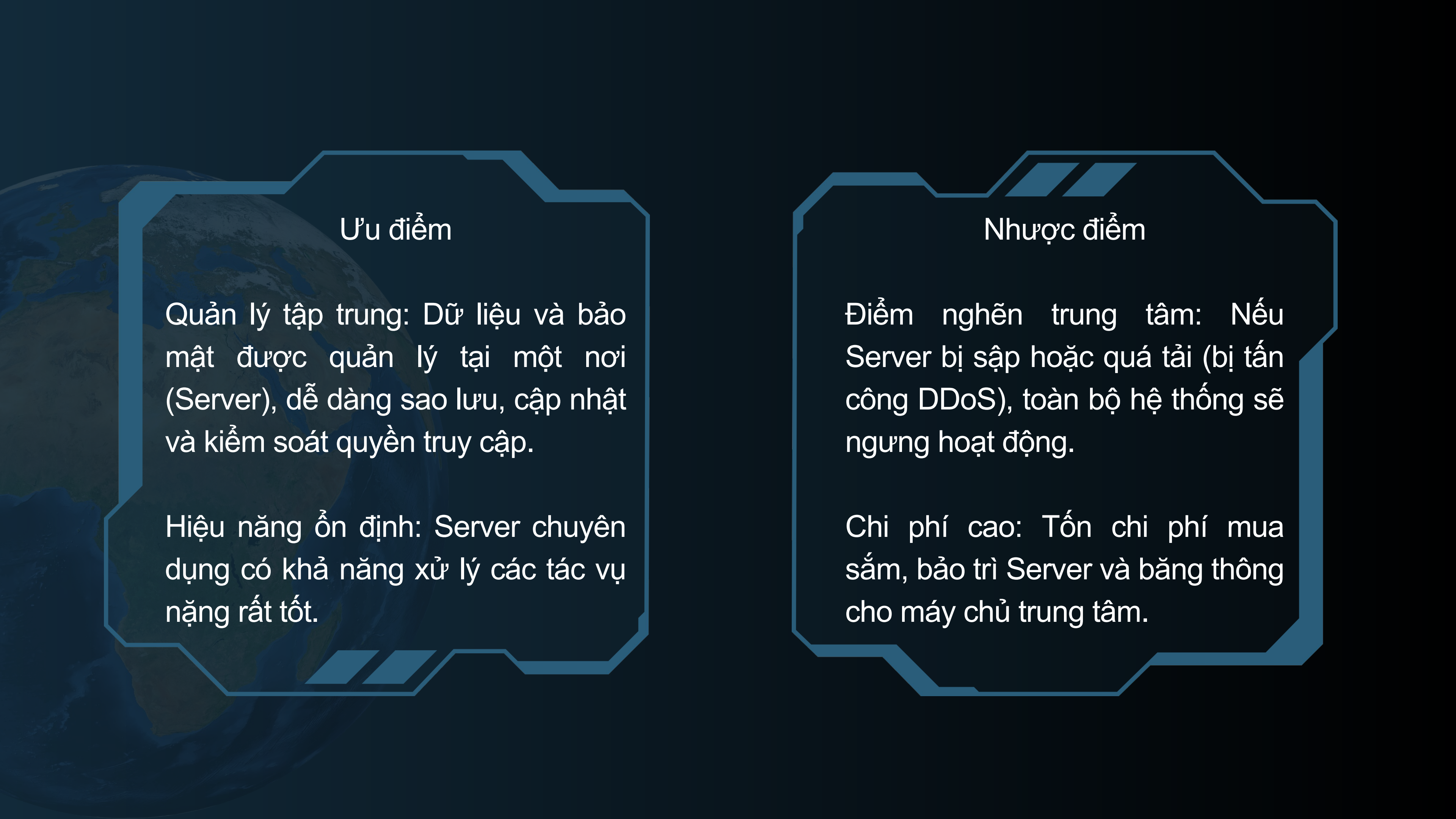
Khái niệm: Là mô hình mạng trong đó các máy tính thành viên được chia làm hai loại:

Server (Máy chủ): Máy tính có cấu hình mạnh, lưu trữ dữ liệu và giữ nhiệm vụ "phục vụ", phản hồi các yêu cầu.

Client (Máy khách): Các máy tính hoặc thiết bị của người dùng (như điện thoại, laptop) gửi yêu cầu (Request) đến Server và nhận kết quả trả về (Response).

Cách thức hoạt động: Quy trình diễn ra theo dạng Yêu cầu - Phản hồi. Client không giao tiếp trực tiếp với nhau mà bắt buộc phải đi qua Server trung gian.





Ưu điểm

Quản lý tập trung: Dữ liệu và bảo mật được quản lý tại một nơi (Server), dễ dàng sao lưu, cập nhật và kiểm soát quyền truy cập.

Hiệu năng ổn định: Server chuyên dụng có khả năng xử lý các tác vụ nặng rất tốt.

Nhược điểm

Điểm nghẽn trung tâm: Nếu Server bị sập hoặc quá tải (bị tấn công DDoS), toàn bộ hệ thống sẽ ngưng hoạt động.

Chi phí cao: Tốn chi phí mua sắm, bảo trì Server và băng thông cho máy chủ trung tâm.

MÔ HÌNH P2P

Khái niệm: Là mô hình không có máy chủ trung tâm. Tất cả các máy tham gia trong mạng (gọi là các Peer hoặc Nút - Node) đều có vai trò và quyền hạn ngang nhau.

Cách thức hoạt động: Mỗi máy vừa đóng vai trò là Client (đi tải dữ liệu từ máy khác) vừa đóng vai trò là Server (chia sẻ dữ liệu của mình cho máy khác). Dữ liệu được băm nhỏ và truyền trực tiếp giữa các máy với nhau.



Ưu điểm

Độ tin cậy cao (Không sợ sập): Nếu một vài máy thoát mạng, hệ thống vẫn hoạt động bình thường nhờ các máy còn lại.

Càng đông càng mạnh: Càng có nhiều người tham gia tải và chia sẻ, tốc độ truyền tải dữ liệu của mạng càng nhanh (do băng thông được chia đều).

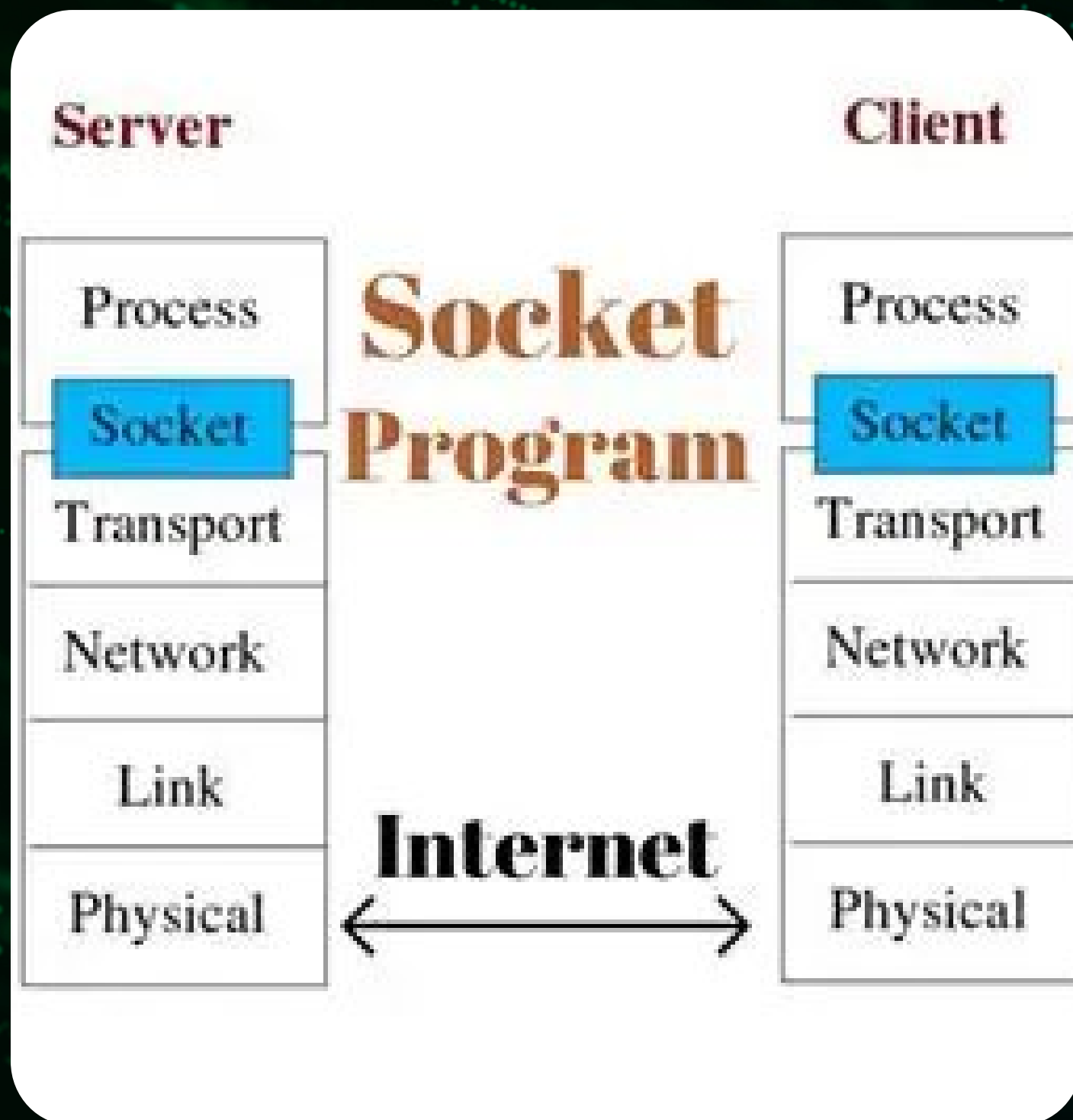
Chi phí thấp: Không tốn tiền mua và duy trì máy chủ trung tâm.

Nhược điểm:

Khó quản lý và bảo mật: Do dữ liệu rải rác ở khắp nơi, rất khó để kiểm soát mã độc hoặc bảo mật thông tin tuyệt đối.

Phụ thuộc vào ý thức người dùng: Nếu không ai chịu "treo máy" để chia sẻ (seed) dữ liệu thì mạng sẽ bị chậm hoặc chết.

SOCKET PROGRAMING



Định nghĩa: Socket là "ổ cắm" giúp hai máy tính có thể kết nối và nói chuyện được với nhau qua mạng.

Để hai máy tìm thấy nhau, một Socket cần có 2 thông tin:

- Địa chỉ IP: Giống như Số điện thoại của nhà bạn (để biết máy nào)
- Cổng (Port): Giống như Số máy nhánh hoặc tên người cần gặp (để biết ứng dụng nào, ví dụ: Port cho Chat, Port cho Game).

Boost.asio



THƯ VIỆN BOOST.ASIO (ASYNCHRONOUS INPUT/OUTPUT)

**ĐÂY LÀ MỘT THƯ VIỆN C++ ĐA
NỀN TẢNG**

**CỰC KỲ NỔI TIẾNG NHỜ KHẢ
NĂNG XỬ LÝ BẤT ĐỒNG BỘ**

**TIẾT KIỆM TÀI NGUYÊN VÀ HỖ
TRỢ TẬN RẰNG**

TCP VÀ UDP

TCP

Cơ chế Bắt tay 3 bước (3-way Handshake):

SYN (Synchronize): Client gửi một lời mời

SYN-ACK (Acknowledgement): Server phản hồi

ACK : Client đã nhận được phản hồi

Tính năng cốt lõi

Đảm bảo thứ tự (Ordering)

Kiểm soát luồng (Flow Control)

Sửa lỗi và Gửi lại (ARQ)

UDP

UDP không có bắt tay 3 bước. Nó cũng không quan tâm bên nhận có đang online hay không. Khi có dữ liệu, ứng dụng chỉ cần đóng gói vào một đơn vị gọi là Datagram (gồm IP, Port và Dữ liệu) rồi đẩy thẳng ra card mạng.

ASYNCHRONOUS PROGRAMMING

Vấn đề của Socket thông thường: "Đóng băng" (Blocking)

Mặc định trong lập trình Socket, các hàm đợi kết nối hoặc đợi nhận dữ liệu là Blocking (Chặn luồng).

Nếu Client chưa gửi gì, Server sẽ đứng im đợi tại dòng code đó.

Hậu quả: Giao diện ứng dụng bị đơ (Not Responding), và Server không thể nhận thêm bất kỳ kết nối của Client nào khác.

Giải pháp Asynchronous: "Rung chuông lấy đồ"

Thay vì ngồi im đứng đợi, lập trình bất đồng bộ chuyển sang cơ chế giao việc và đợi sự kiện:

Hệ điều hành có thể đi làm việc của nó khi nào có gói tin thì sẽ quay lại làm tiếp công việc này

Không tốn tài nguyên như muti threading và giữ cho ứng dụng không bị đơ vì blocking

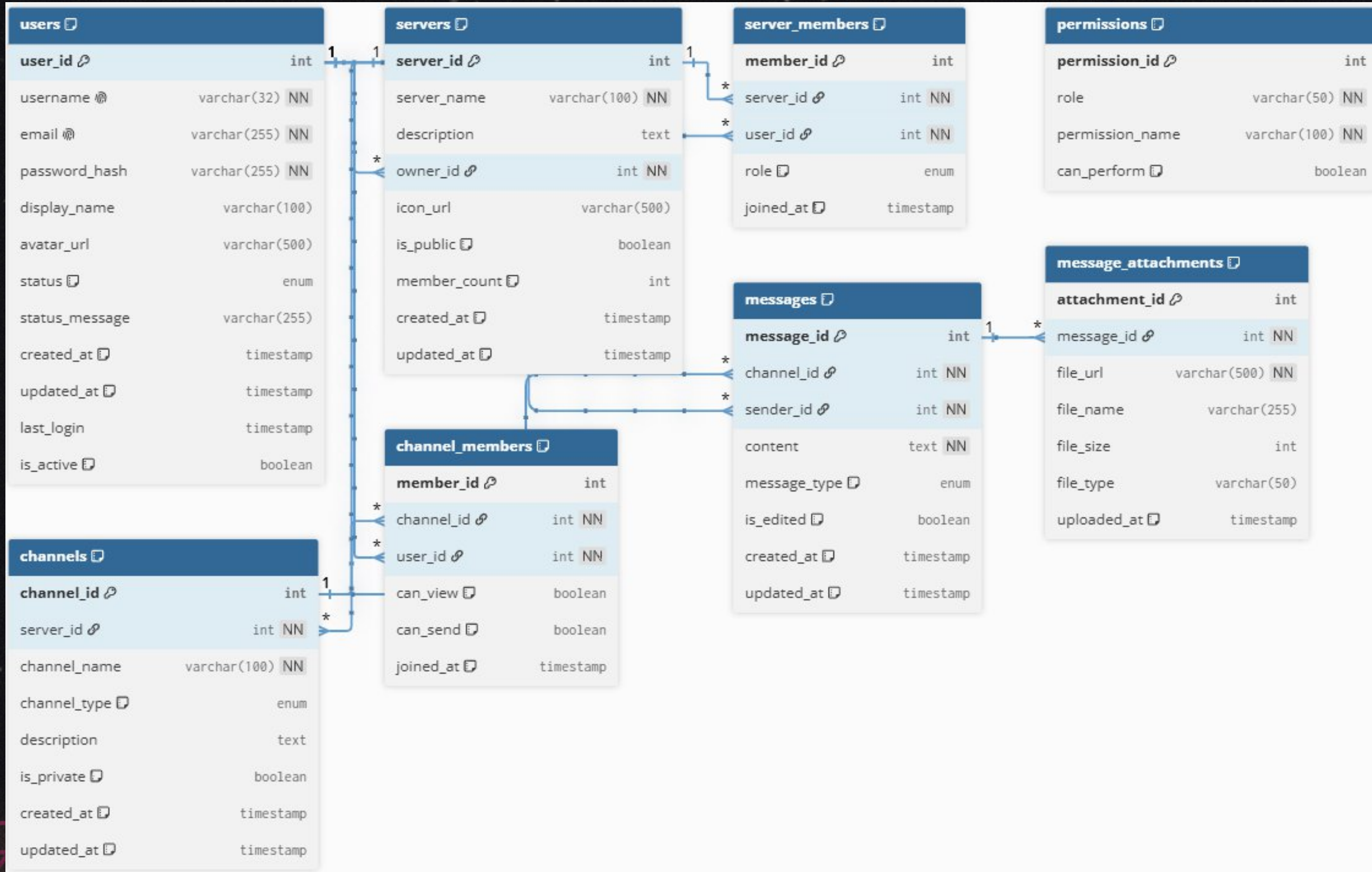
2 KIẾN TRÚC & CHI TIẾT DỰ

ÁN

```
App_Chatties
├── src/
│   ├── server/
│   │   ├── core/
│   │   │   ├── asio_server.cpp
│   │   │   ├── asio_server.h
│   │   │   ├── connection_manager.cpp
│   │   │   └── connection_manager.h
│   │   ├── handlers/
│   │   │   ├── message_handler.cpp
│   │   │   ├── message_handler.h
│   │   │   ├── user_handler.cpp
│   │   │   └── user_handler.h
│   │   ├── media/
│   │   │   ├── audio_processor.cpp
│   │   │   ├── audio_processor.h
│   │   │   ├── video_processor.cpp
│   │   │   └── video_processor.h
│   │   └── main.cpp
│   └── client/
│       ├── ui/
│       │   ├── main_window.cpp
│       │   ├── main_window.h
│       │   ├── chat_widget.cpp
│       │   └── chat_widget.h
│       ├── network/
│       │   ├── socket_client.cpp
│       │   ├── socket_client.h
│       │   └── protocol_handler.cpp
│       ├── media/
│       │   ├── audio_manager.cpp
│       │   ├── audio_manager.h
│       │   ├── video_manager.cpp
│       │   └── video_manager.h
│       └── main.cpp
└── common/
    ├── protocol/
    │   ├── message_protocol.h
    │   └── packet_definitions.h
    ├── utils/
    │   ├── logger.cpp
    │   ├── logger.h
    │   ├── serializer.cpp
    │   └── serializer.h
    ├── constants.h
    └── database/
        ├── schemas/
        │   ├── users.sql
        │   ├── servers.sql
        │   ├── channels.sql
        │   ├── messages.sql
        │   └── permissions.sql
        └── migrations/
            └── README.md
```



2.1 DATA BASE



USER TABLE

users	
user_id	int
username	varchar(32) NN
email	varchar(255) NN
password_hash	varchar(255) NN
display_name	varchar(100)
avatar_url	varchar(500)
status	enum
status_message	varchar(255)
created_at	timestamp
updated_at	timestamp
last_login	timestamp
is_active	boolean

MESSAGE TABLE

messages	
message_id	int
channel_id	int NN
sender_id	int NN
content	text NN
message_type	enum
is_edited	boolean
created_at	timestamp
updated_at	timestamp

SERVER AND SERVER MEMBER

servers		
server_id		int
server_name	varchar(100)	NN
description		text
owner_id		int NN
icon_url	varchar(500)	
is_public		boolean
member_count		int
created_at		timestamp
updated_at		timestamp

server_members		
member_id		int
server_id		int NN
user_id		int NN
role		enum
joined_at		timestamp

CHANNEL AND CHANNEL MEMBER

channels		
channel_id	int	
server_id	int	NN
channel_name	varchar(100)	NN
channel_type	enum	
description	text	
is_private	boolean	
created_at	timestamp	
updated_at	timestamp	

channel_members		
member_id	int	
channel_id	int	NN
user_id	int	NN
can_view	boolean	
can_send	boolean	
joined_at	timestamp	

MESSAGE_ATTACHMENTS AND PERMISSIONS

message_attachments

attachment_id 	int
message_id 	int NN
file_url	varchar(500) NN
file_name	varchar(255)
file_size	int
file_type	varchar(50)
uploaded_at 	timestamp

permissions

permission_id 	int
role	varchar(50) NN
permission_name	varchar(100) NN
can_perform 	boolean

2.2 CHỨC NĂNG ĐĂNG KÍ VÀ ĐĂNG NHẬP

The image displays two side-by-side screenshots of a web application titled 'Chatties'. The left screenshot shows the 'Login' form, and the right screenshot shows the 'Register' form. Both forms are presented in a dark-themed window with a light blue header bar.

Left Screenshot (Login Form):

- Header: 'Chatties' (white text on a dark background).
- Form Header: A dark bar with two tabs: 'Login' (active, highlighted with a blue underline) and 'Register'.
- Fields: Two light gray input fields labeled 'Username' and 'Password'.
- Button: A large, rounded blue button labeled 'Login'.

Right Screenshot (Register Form):

- Header: 'Chatties' (white text on a dark background).
- Form Header: A dark bar with two tabs: 'Login' and 'Register' (active, highlighted with a blue underline).
- Fields: Four light gray input fields labeled 'Username', 'Email', 'Display name', and 'Password', followed by a 'Confirm password' field.
- Button: A large, rounded blue button labeled 'Register'.

2.2 CHỨC NĂNG ĐĂNG KÍ VÀ ĐĂNG NHẬP

Luồng sự kiện: Nút Đăng ký

Bước 1 : Người dùng nhấn nút Đăng ký -> Kích hoạt sự kiện

Bước 2 : Ứng dụng kiểm tra tính hợp lệ dữ liệu (tên tài khoản, mật khẩu nhập lại).

Bước 3 : Gọi hàm xử lý, đóng gói thông tin đăng ký thành dữ liệu JSON/Byte Stream.

Bước 4 : Gửi gói tin qua TcpSocket tới Server -> Chờ phản hồi.

Bước 5 : Nhận phản hồi từ Server -> Phát tín hiệu (Signal) thành công/thất bại -> Giao diện (UI) chuyển màn hình hoặc báo lỗi.

2.2 CHỨC NĂNG ĐĂNG KÍ VÀ ĐĂNG NHẬP

Luồng sự kiện: Nút Đăng nhập

Bước 1 : Người dùng nhập thông tin và nhấn nút Đăng nhập

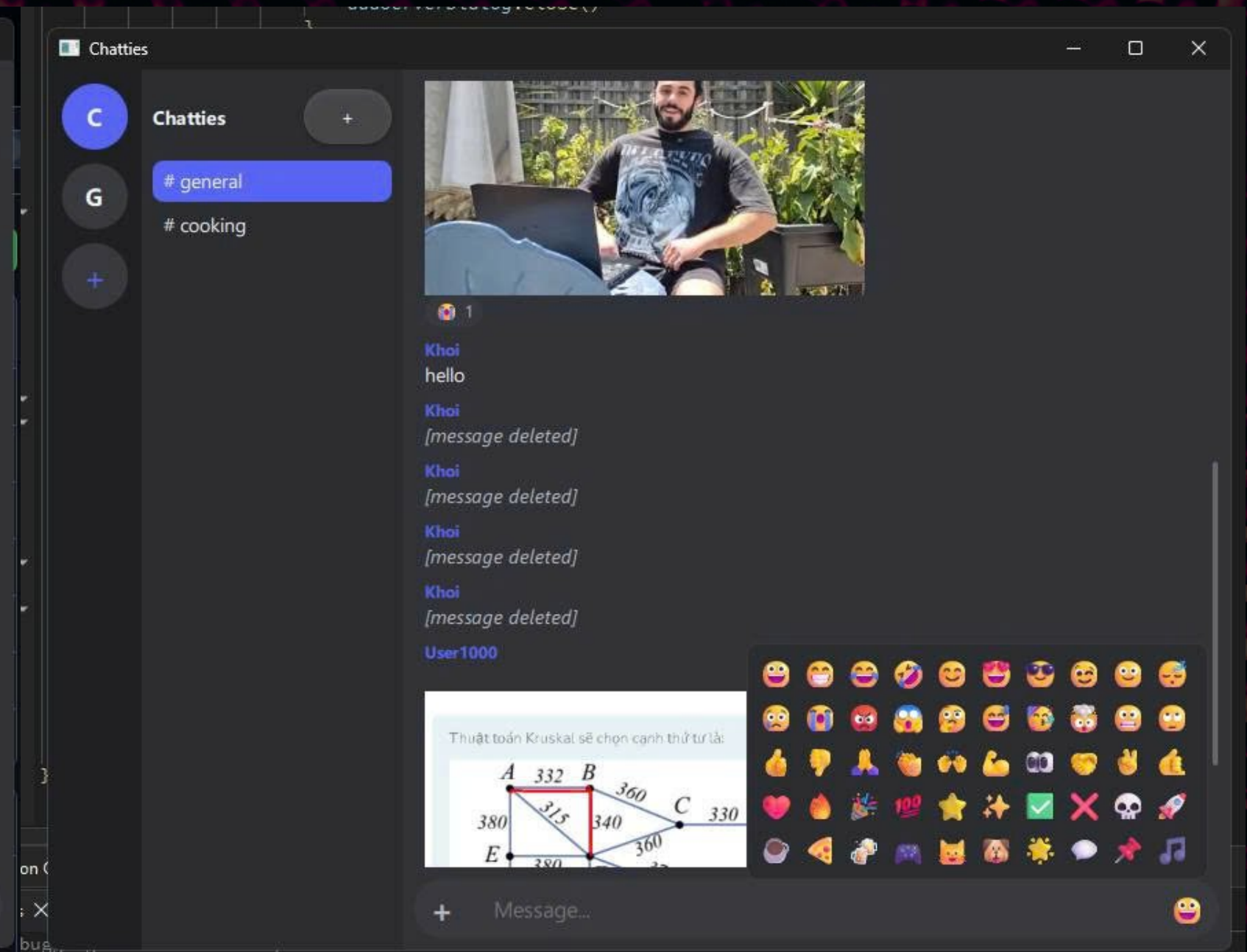
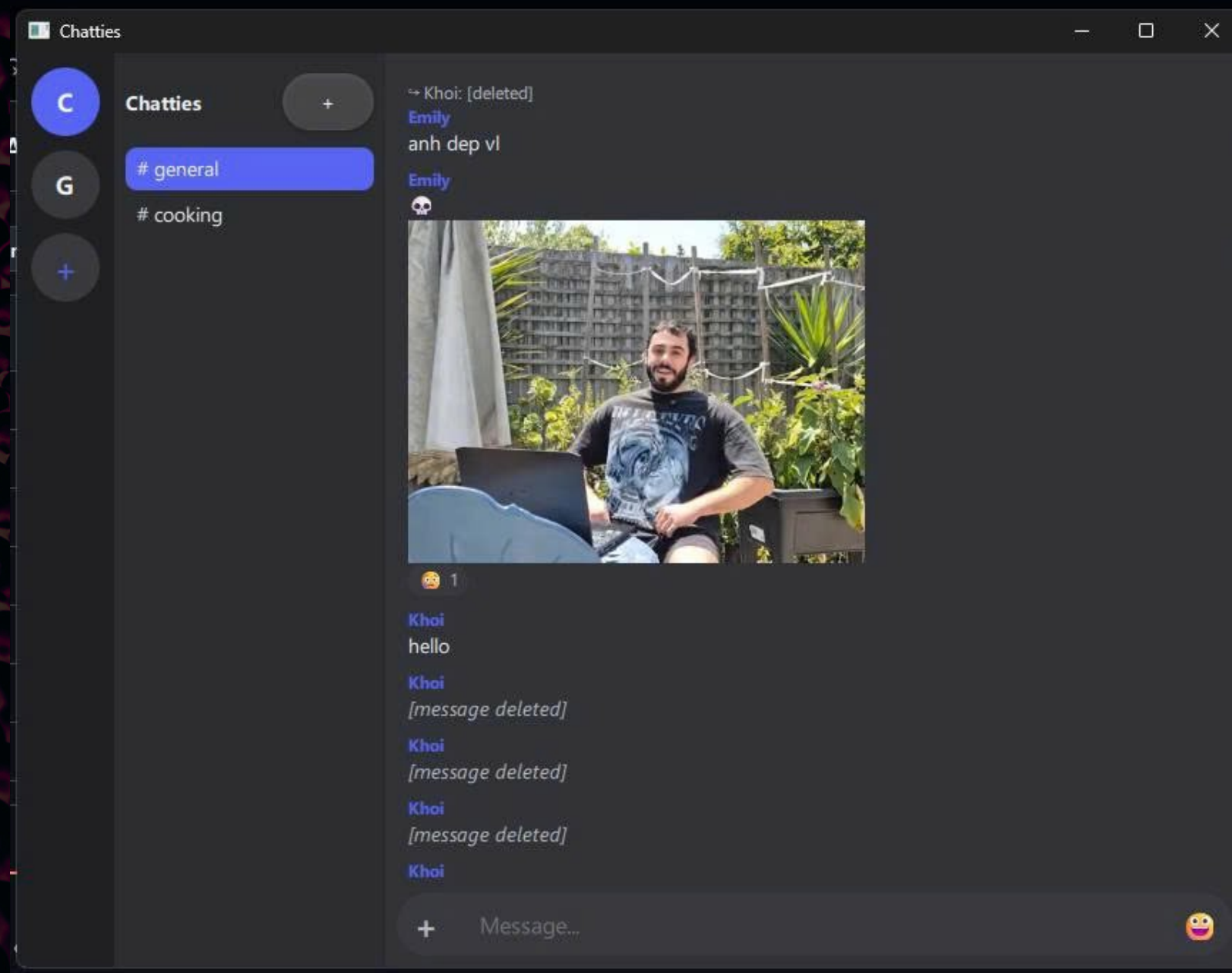
Bước 2 : Xác thực định dạng -> Gọi hàm login() kết nối tới Socket của Server.

Bước 3 : Truyền dữ liệu tài khoản mã hóa qua mạng ->UI hiển thị trạng thái chờ

Bước 4 : Kích hoạt sự kiện trên Server để phân tích dữ liệu->Server phản hồi kết quả

Bước 5 : Client nhận tín hiệu đăng nhập thành công -> Giao diện chuyển từ màn hình chờ sang Giao diện Chat chính.

2.3 CHỨC NĂNG CHAT NHÓM, 1-1



Bước 1

Người dùng nhập văn bản và nhấn nút Gửi (hoặc phím Enter).

Bước 2

Kiểm tra nội dung tin nhắn (đảm bảo không bị trống).

Bước 3

Đóng gói tin nhắn

Bước 4

Gửi gói tin qua Socket -> Đồng thời xóa trống ô nhập liệu và cập nhật tin nhắn mới lên màn hình hiển thị

Luồng sự kiện: Nút Gửi tin nhắn (Send)

KIẾN TRÚC XỬ LÝ BẤT ĐỒNG BỘ

Hệ thống sử dụng mô hình Kiến trúc hướng sự kiện (Event-driven Architecture) dựa trên 3 vòng lặp sự kiện (Event Loop) độc lập, không sử dụng Message Queue trung tâm.

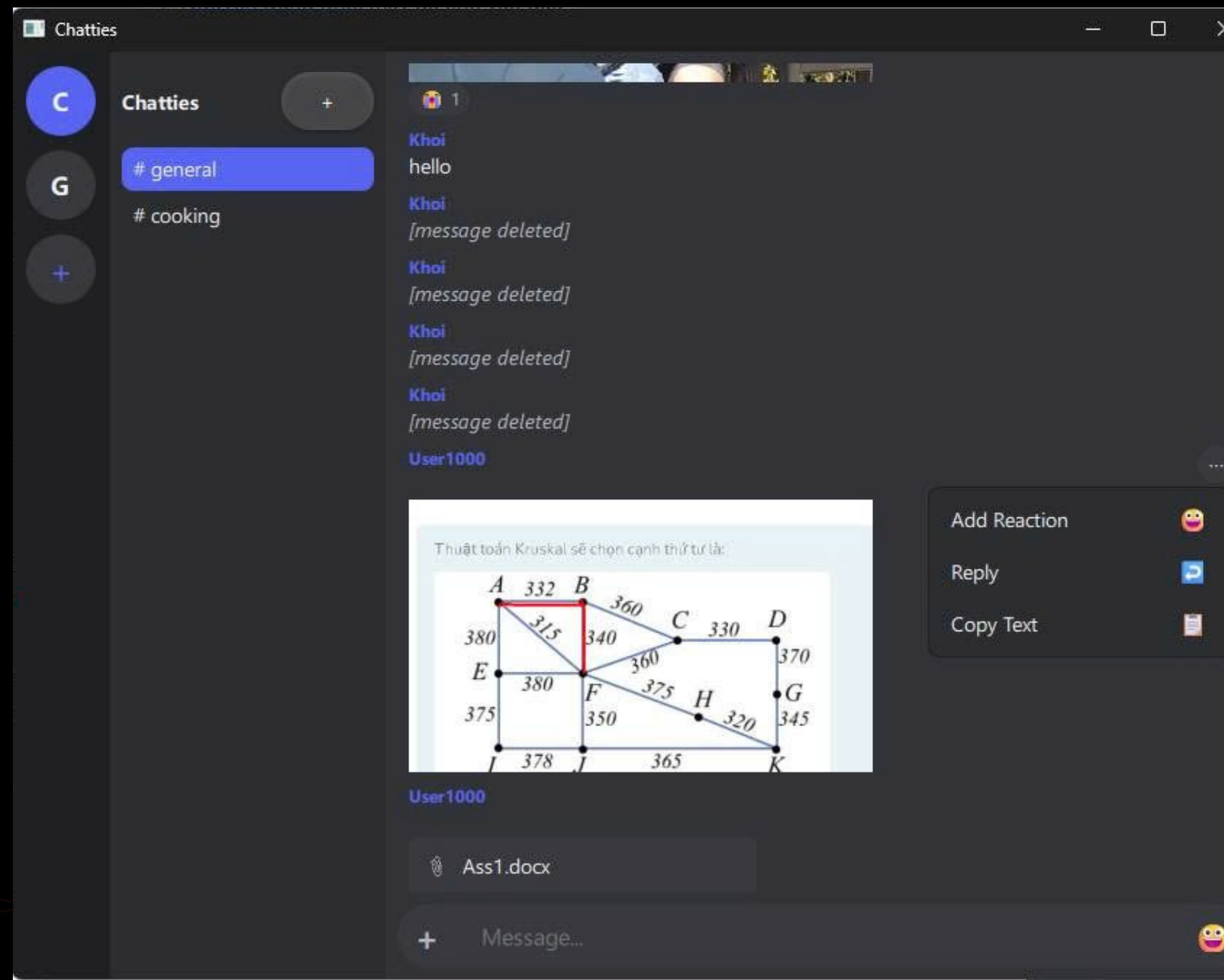
I/O Bất đồng bộ: Server sử dụng vòng lặp callback `async_read_until` để nhận dữ liệu từ Socket và `async_write` để đẩy dữ liệu ra theo cơ chế Fire-and-forget (gửi và quên, không có hàng đợi ghi).

Xử lý Nghiệp vụ Đồng bộ (Điểm nghẽn hệ thống):

Khi nhận lệnh `message.forward`, hàm `handle_line()` được thực thi ngay lập tức bên trong luồng callback.

Toàn bộ quá trình: Kiểm tra quyền (`can_access`), truy vấn DB, sao chép metadata đính kèm và quét danh sách kết nối để Broadcast đều chạy tuần tự (blocking) trên Event Loop duy nhất này.

RELY MESSAGE



QUYỀN NĂNG CHUYỂN TIẾP

Điều kiện

Quyền kê

Quyền kê

Để một tin nhắn được forward thành công, hệ thống bắt buộc phải kiểm tra qua 4 điều kiện sau:

2.4 CHỨC NĂNG GỬI

FILE

Mô hình: Upload Hybrid (HTTP + WebSocket)

LUỒNG XỬ LÝ:

CLIENT KIỂM TRA FILE -> UPLOAD DẠNG BINARY QUA HTTP POST.

SERVER LƯU TRỮ -> TRẢ VỀ METADATA (ID, URL, KÍCH THƯỚC).

CLIENT GỬI TIN NHẮN KÈM METADATA QUA WEBSOCKET (

Các giới hạn hệ thống

Space

Dung lượng tối đa: 5 MB / file.

Number

Số lượng tối đa: 10 file / tin nhắn.

Validation

Cơ chế kiểm tra: Thực hiện song song tại cả Client (UI/UX) và Server (Security).

Flexibility

Tính linh hoạt: Chấp nhận tin nhắn chỉ có file (trường text được phép để trống).

2.5 CHỨC NĂNG AVATAR

3 CƠ CHẾ HIỂN THỊ:

Avatar là chuỗi URL (avatar_url) lưu trong DB (users.avatar_url)

01

Mặc định (Không có ảnh): Hiển thị Vòng tròn màu + Chữ cái đầu của tên

02

Avatar Preset: Sử dụng 4 ảnh nhân vật có sẵn trong app

03

Avatar Tự Upload: Ảnh do người dùng tải lên ứng dụng cắt tròn

2.6 CƠ CHẾ BẢO MẬT

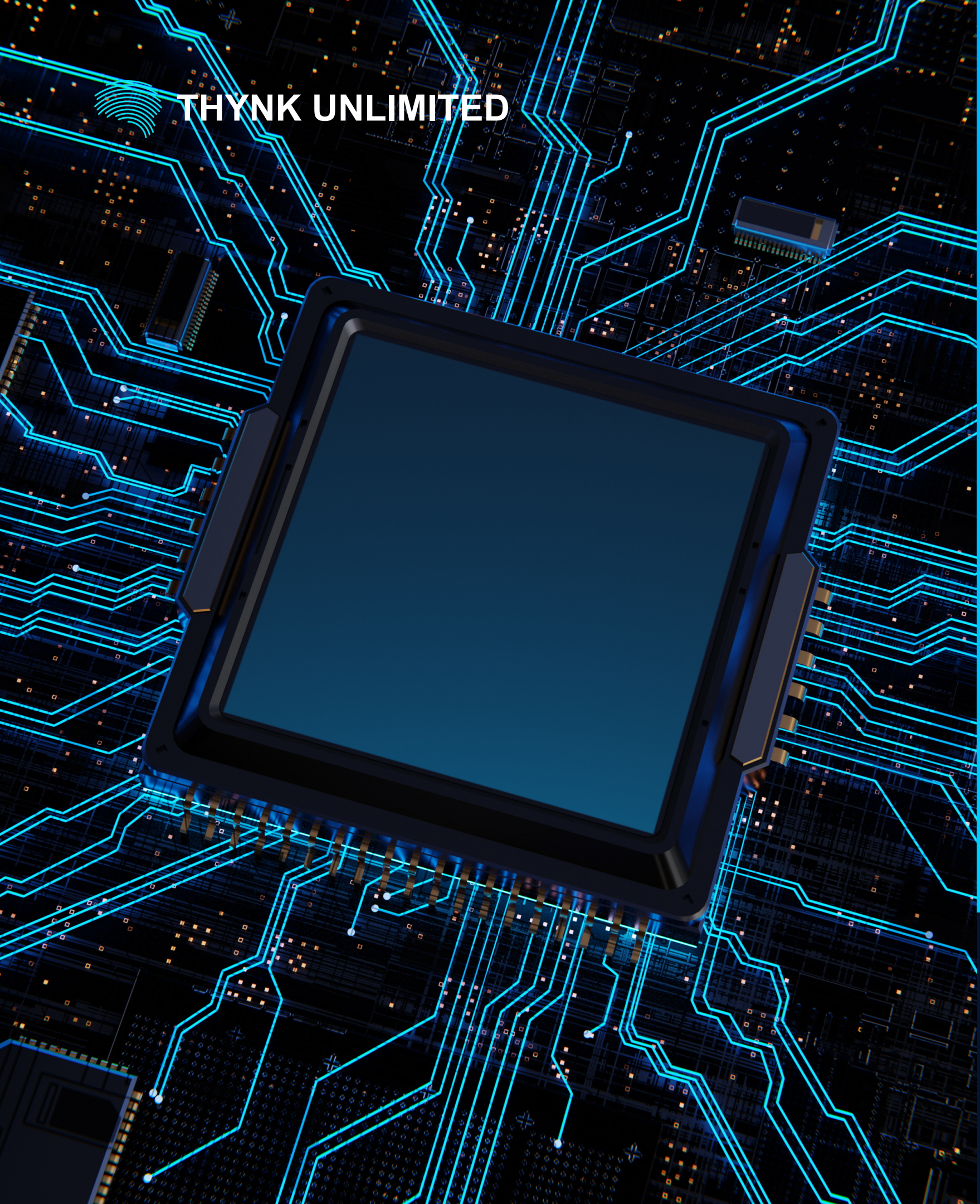
ARGON2/LIBSODIUM

BẢN CHẤT: KẾT HỢP THUẬT TOÁN BẮM ARGON2 (MẠNH NHẤT HIỆN NAY) VÀ THƯ VIỆN MÃ HÓA LIBSODIUM

CƠ CHẾ (MEMORY-HARD): BUỘC TIÊU TÓN MỘT LƯỢNG RAM CỐ ĐỊNH KHI BẮM MẬT KHẨU.

ƯU ĐIỂM: KHÁNG HOÀN TOÀN CÁC CUỘC TẤN CÔNG BẰNG KHÓA HÀNG LOẠT BẰNG SIÊU MÁY TÍNH HOẶC CHIP ĐỒ HỌA (GPU/ASIC).





THYNK UNLIMITED

TỔNG KẾT VÀ HƯỚNG PHÁT TRIỂN

Về mặt tính năng cốt lõi

Hệ thống đã hiện thực hóa được cấu trúc không gian giao tiếp tương tự Discord bao gồm: tạo tài khoản/giao diện người dùng, hệ thống các máy chủ (Servers), các kênh chat (Channels) riêng biệt. Người dùng có thể tham gia vào các không gian chung và trò chuyện thời gian thực.

Về mặt kỹ thuật & Bảo mật:

Sử dụng giải pháp mã hóa dữ liệu trên đường truyền để bảo vệ nội dung tin nhắn và thông tin tài khoản của người dùng, ngăn chặn các nguy cơ nghe lén (Sniffing) dữ liệu.

Xử lý đồng bộ và tối ưu hóa kết nối đa luồng (Multi-threading) hoặc Socket để đảm bảo luồng tin nhắn giữa các thành viên trong kênh được cập nhật ngay lập tức mà không bị delay.

HƯỚNG PHÁT TRIỂN

01 VIDEO CALL

Nâng cấp hệ thống gọi điện/gọi video dụng giao thức WebRTC

01 ENCRPTION

Bổ sung tính năng mã hóa đầu cuối cho các cuộc trò chuyện riêng tư.

01 CND

Tích hợp mạng phân phối nội dung để tối ưu tốc độ tải media/avatar khi số lượng người dùng tăng lên.

The background is a deep purple with various geometric shapes and lines. Two stylized, segmented robotic hands in shades of purple and blue are positioned on the left and right sides, palms facing each other. In the center, there's a large, glowing purple diamond shape with lines radiating from it towards the bottom. The overall aesthetic is futuristic and high-tech.

THANK YOU!

We appreciate your time and attention.